

Basic Software Engineering Workflows

Reproducibility, Isolation, and Production-Grade Code
Hygiene

TOPIC 01

The ``Works on My Machine'' Curse: Environments

Dependency isolation, venv/uv internals, sys.path resolution, and lockfiles.

Cognitive Objectives

By the end of this topic, learners will be able to:

- **Diagnose dependency collisions** in global Python `site-packages` and understand `sys.path` resolution.
- **Inspect virtual environment internals** (isolated binaries, symlinks, `pyenv.config`, and isolated packages).
- **Author modern packaging specifications** using declarative `pyproject.toml` (PEP 621 / PEP 517).
- **Enforce zero-drift reproducibility** with deterministic lockfiles (`uv.lock`) in production cloud pipelines.

The Dependency Collision Catastrophe

The Collision: Global Package Overwrites

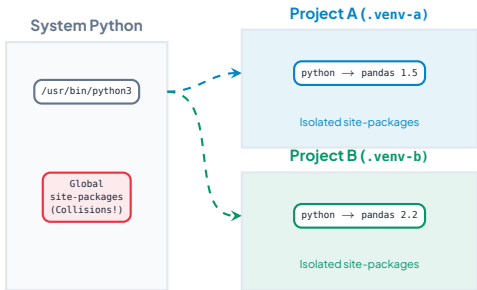
- **Legacy Model API (Project A):** Requires `pandas==1.5.3` and `numpy<1.24`.
- **New GenAI Pipeline (Project B):** Requires `pandas>=2.2.0` and `numpy>=1.26`.
- Running `pip install pandas==2.2` globally silently overwrites the files in `/usr/lib/python3.11/site-packages/`.
- **Production Outage:** Project A's production inference server crashes with `AttributeError`!

The "Aha!" Discovery: Path Sandboxing

- A Virtual Environment is not a VM or container; it is simply a **directory tree with a redirected `sys.prefix`!**
- CPython checks `pyvenv.cfg` on startup.
- `sys.path` prepends the local `.venv/lib/python/site-packages/` ahead of global directories.
- Multiple isolated environments coexist on the same machine without mutual interference.

Never execute `pip install`
Important outside an active virtual
environment.

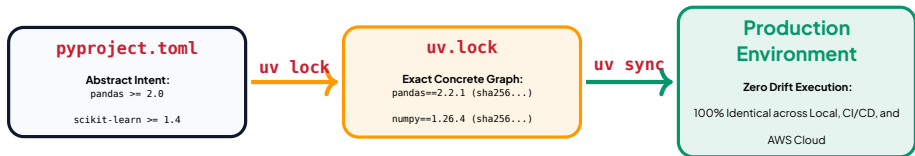
Hero Visualization: Virtual Environment Sandboxing



How CPython Resolves Imports

- **pyvenv.cfg:** Defines `home = /usr/bin` and `include-system-site-packages = false`.
- **sys.executable:** Points to the local environment binary `./venv/bin/python`.
- **sys.path Order:**
 1. Current Working Directory ("").
 2. Virtualenv **site-packages**.
 3. Python Standard Library.

Hero Visualization: Modern Packaging & Lockfile Pipeline



Important

Commit **uv.lock** to Git to guarantee bit-for-bit reproducible ML environments.

Production Checkpoint: High-Speed Operations with uv

Modern Cloud Python Toolchain

- Written in Rust, **uv** is 10 – 100× faster than standard **pip** and **virtualenv**.
- Global hardlink caching avoids copying duplicate wheels across projects.
- Replaces **pip**, **pip-tools**, **virtualenv**, **poetry**, and **pyenv**.

Important Sub-millisecond dependency synchronization in production.

Essential Production Commands

```
1 # 1. Create lightning-fast virtualenv
2 uv venv --python 3.11
3
4 # 2. Add dependencies with instant resolution
5 uv add pandas numpy scikit-learn
6
7 # 3. Deterministic sync from locked graph
8 uv sync --frozen
```

TOPIC 02

Time Travel & Parallel Realities: Git Internals

Directed Acyclic Graphs, SHA-1 snapshots, branching, and merge conflict surgery.

Cognitive Objectives

By the end of this topic, learners will be able to:

- **Model Git internals** as a content-addressable directed acyclic graph (DAG) of Blobs, Trees, and Commits.
- **Deconstruct branch pointers** as lightweight 41-byte SHA-1 text files rather than heavy data copies.
- **Choose between 3-Way Merging and Rebasing** to maintain clear, auditable collaborative commit histories.
- **Resolve merge conflicts defensively** without losing critical upstream data science code or model weights.

The Mental Model: Diffs vs Snapshots

The Intuitive Misconception: Delta Diffs

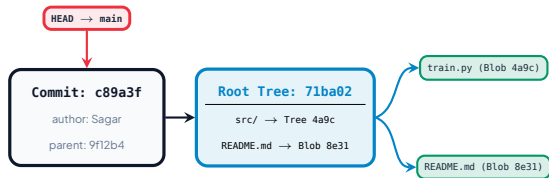
- Common belief: Git stores a base file plus a series of incremental text diffs (+10 lines, -3 lines).
- **Flaw:** Reconstructing a file from 10,000 commits would require slow sequential diff playback.
- How does `git checkout` switch across branches in 2 milliseconds?

The "Aha!" Discovery: Directed Acyclic Graph

- **Git stores snapshots, not diffs!**
- Every commit points to a complete root **Tree** snapshot of the entire repository.
- Unchanged files share identical **Blob** pointers via content-hash deduplication (SHA-1 / SHA-256).
- Branches are merely 41-byte text pointers moving along the commit DAG!

Important Git is a content-addressable key-value object database.

Hero Visualization: The Git Object Model (Blobs, Trees, Commits)

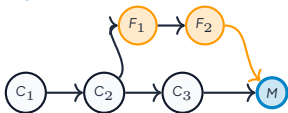


Content-Addressed Storage

- **Blob:** Raw binary payload of file contents. File names and permissions are stored in Trees, not Blobs.
- **Tree:** Directory mapping holding file names, permissions, and hash pointers to child Trees / Blobs.
- **Commit:** Author metadata, timestamp, commit message, and pointer to parent commit hash.

Hero Visualization: 3-Way Merge vs Rebase Topologies

3-Way Merge: Preserves Branch Divergence



Topology: Non-linear history. Retains exact chronological branching timestamps and diamond merge commit M .

Rebase: Clean Linear Narrative



Topology: Completely linear DAG. Replays feature commits on top of the latest **main**. Eliminates messy diamond merge clutter.

Production Checkpoint: Reset vs Revert Safety Protocol

Local Sandbox: `git reset`

- **Action:** Rewinds branch ref to an earlier commit and discards unpushed commits.
- **Use Case:** Local feature branch before pushing upstream.
- **Golden Rule:** **Never** reset commits that have already been pushed to shared remote branches!

Shared Remote: `git revert`

- **Action:** Creates a new forward-moving commit that applies the exact inverse patch of the target commit.
- **Use Case:** Rolling back broken production deployments without rewriting shared history.
- Preserves DAG integrity for all team collaborators.

Use `git revert` for shared
Important! branches to protect
collaboration.

TOPIC 03

Silent Failure Prevention: Pytest & Quality

Deterministic unit testing, test fixtures, parametrization, and ruff linting.

Cognitive Objectives

By the end of this topic, learners will be able to:

- **Prevent silent data corruption** in ML feature pipelines using deterministic unit tests.
- **Structure modular test fixtures** and manage setup/teardown lifecycles using dependency injection.
- **Scale test coverage systematically** with `@pytest.mark.parametrize` across critical edge cases.
- **Automate code hygiene** using ultra-fast static linters (`ruff`) and type checkers (`mypy`).

The Silent Failure Disaster

The Hazard: Code Runs, Data Dies

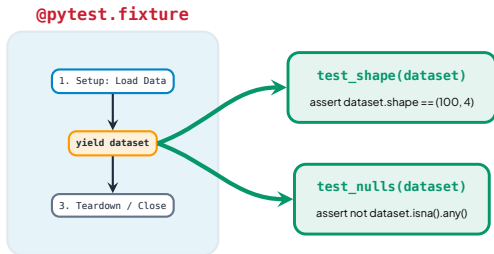
- A feature normalization function contains an off-by-one division error or silently fills missing values with 0.0.
- The script exits with exit code 0 without raising any exceptions!
- Model trains on corrupted features and deploys to production.
- **Business Impact:** Millions in mispriced loans before anyone notices!

The "Aha!" Discovery: Executable Contracts

- Unit tests act as **executable specifications** that verify data transformations across known invariants.
- Test for shape preservation, non-negativity, mathematical bounds, and null rejection.
- Defensive assertions catch drift before weights are fitted.

Important Crashing early is a feature; silent data corruption is a disaster.

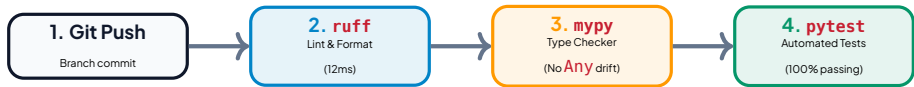
Hero Visualization: Fixture Lifecycle & Injection



Dependency Injection Mechanics

- **Auto-Wiring:** Pytest passes fixture return values into test functions whose parameter names match the fixture name.
- **Deterministic Isolation:** Each test function receives a fresh copy of the resource.
- **Scopes:** `function` (default), `module` (once per file), `session` (once per test run).

Hero Visualization: Modern CI/CD Quality Gate Pipeline



Important

Automated pre-commit hooks and CI pipelines stop defective code before it reaches production.

Production Checkpoint: Parametrized Test Suite

Parametrization Invariants

- Automatically executes the same test across multiple input/output tuples.
- Tests nominal inputs alongside zero vectors, empty matrices, and NaN edge cases.
- Generates individual test reports for each parameterized test case.

Important Edge case testing eliminates production surprise failures.

Parametrized Test Code

```
1 import pytest
2 import numpy as np
3
4 @pytest.mark.parametrize("vec, expected_norm", [
5     (np.array([3.0, 4.0]), 5.0),
6     (np.array([0.0, 0.0]), 0.0),
7     (np.array([1.0, 1.0, 1.0]), np.sqrt(3)),
8 ])
9 def test_vector_norm_calculation(vec, expected_norm):
10     result = np.linalg.norm(vec)
11     assert np.isclose(result, expected_norm, atol=1e-6)
```

Session Summary

01 The “Works on My Machine” Curse: Environments

Dependency isolation, venv/uv internals, sys.path resolution, and lockfiles.

02 Time Travel & Parallel Realities: Git Internals

Directed Acyclic Graphs, SHA-1 snapshots, branching, and merge conflict surgery.

03 Silent Failure Prevention: Pytest & Quality

Deterministic unit testing, test fixtures, parametrization, and ruff linting.

Professional Program in Enterprise Data Science & Agentic AI with AWS — Modular Session Architecture